SISTEMA VISUAL E ESPECIFICAÇÃO DE INTERFACE

Deep Ocean Interface.

Uma linguagem visual consistente para uma ferramenta de desenvolvimento profissional: modular, técnica e orientada ao trabalho.

Dark-first · Modular · Desktop-first · Adaptável



Referência visual fornecida — usada para orientar a linguagem visual, não para fixar conteúdo ou dados circunstanciais.

GUIA DE REFERÊNCIA

Mapa do documento

Fundamentos, arquitetura, componentes e especificações de cada tela em um único sistema visual.

01 **Fundamentos da identidade visual**
Conceito, cores, tipografia, geometria e profundidade

02 **Arquitetura global**
Barra superior, navegação, canvas e painéis

03 **Componentes fundamentais**
Botões, campos, cards, ícones e estados

04 **Especificação das telas**
Chat, Tasks, Context, Tools, Providers, Settings, Prompt e Specifications

05 **Navegação e consistência**
Elementos persistentes, contextuais e atalhos

06 **Microinterações**
Tempos, transições e feedback

07 **Acessibilidade e ergonomia**
Contraste, teclado, foco e movimento

08 **Especificações técnicas**
Tokens, CSS e organização de arquivos

09 **Qualidade e validação**
Critérios objetivos e checklist

10 **Direção final**
Princípios duradouros e validações pendentes

Importante

Medidas e comportamentos descritos são uma proposta de design system. Estados de execução, integrações e ações devem refletir sempre as capacidades reais do produto.

Como usar este documento

Este documento converte a linguagem visual da referência em um conjunto de regras reutilizáveis para o Oh My Shark. Ele cobre fundamentos visuais, estrutura da aplicação, componentes, telas, interações, acessibilidade e recomendações técnicas. A intenção é que novas telas possam ser desenvolvidas sem inventar uma linguagem visual diferente a cada implementação.

É importante distinguir três categorias de informação:

- **Regra de design:** orientação visual ou comportamental recomendada para preservar a consistência.
- **Valor de referência:** medida, cor ou breakpoint inicial, sujeito a ajustes após implementação e testes.
- **Capacidade do produto:** comportamento que depende do código, dos provedores, das ferramentas ou das permissões realmente disponíveis. O design system não presume que uma função exista apenas porque é mostrada como exemplo.

Os textos, nomes de modelos, mensagens, tarefas, caminhos de arquivos, ferramentas, indicadores de uso e estados da captura original são conteúdo circunstancial. Eles não devem ser incorporados ao layout como elementos fixos. O sistema precisa acomodar conteúdo variável, ausente, adicional ou indisponível sem quebrar a composição.

Princípio central

> **A referência define a linguagem visual; o conteúdo da referência não define a estrutura obrigatória do produto.**

O Oh My Shark deve se comportar como um ambiente de desenvolvimento profissional: sóbrio, técnico, responsivo e claro. A área de trabalho deve receber a maior parte da atenção. A navegação deve ser previsível. As cores devem orientar a ação, não competir com o conteúdo.

---

1. Fundamentos da identidade visual

1.1 Conceito: Deep Ocean Interface

A identidade visual pode ser entendida como um oceano digital profundo. O fundo escuro reduz o ruído visual; diferentes tons de azul-marinho distinguem as superfícies; bordas finas definem os limites; o azul elétrico direciona a atenção para foco, seleção e ações relevantes. O resultado precisa transmitir precisão, controle, velocidade e profundidade técnica, sem parecer um painel administrativo genérico ou uma cópia de outro produto.

Personalidade visual

- **Técnica:** a interface acomoda comandos, código, logs e informações operacionais sem torná-los estranhos ao restante do produto.
- **Sóbria:** o contraste é intencional; brilhos e cores saturadas são usados com moderação.

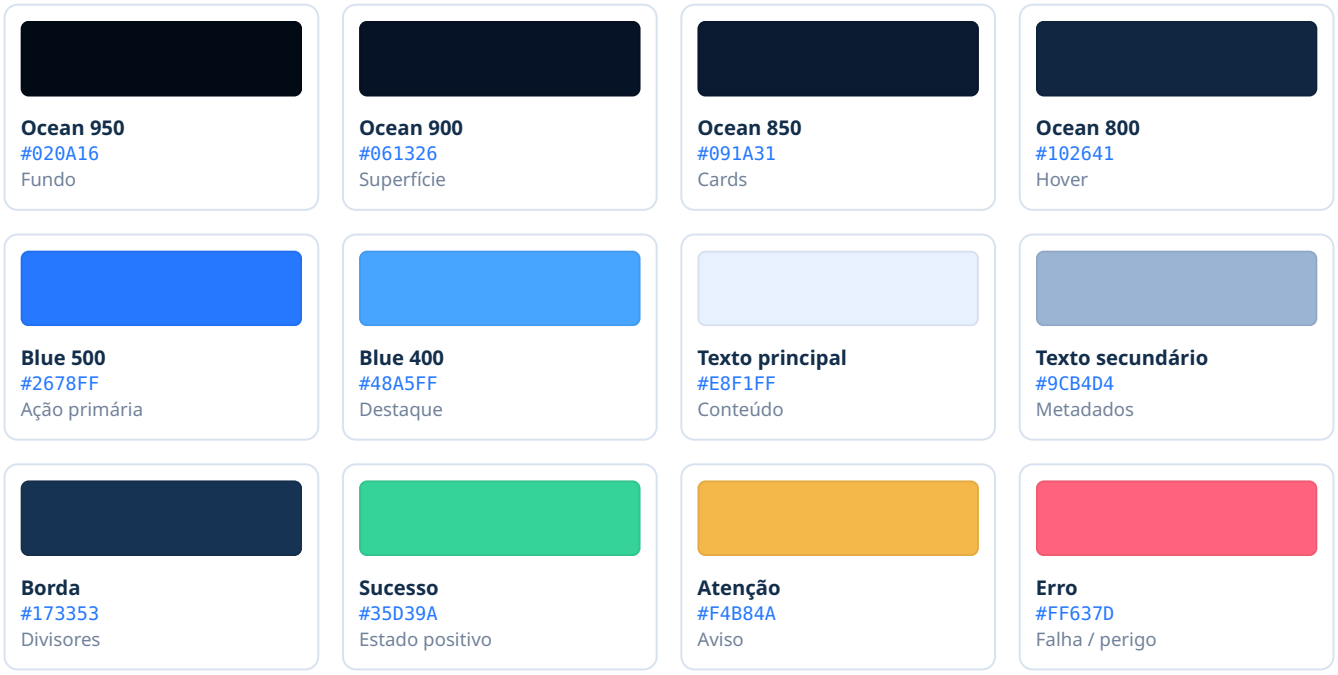
- **Inteligente:** a hierarquia mostra o que é importante agora e reduz a presença de metadados secundários.
- **Orientada ao trabalho:** a interface favorece leitura, edição, decisão e acompanhamento de tarefas.
- **Flexível:** recursos opcionais aparecem somente quando relevantes e disponíveis.

O que evitar

Evitar neon permanente, gradientes decorativos em toda a interface, sombras intensas em cada card, ícones com estilos misturados, painéis sempre preenchidos e cores sem semântica. A personalidade deve vir da combinação de forma, contraste, ritmo e comportamento, não de efeitos incessantes.

1.2 Paleta cromática

A paleta trabalha em camadas: profundidade para as superfícies, azul para a ação e cores semânticas para comunicar estados.



Os códigos a seguir são uma paleta de referência inspirada na captura. Eles devem ser centralizados em tokens, para que ajustes futuros não exijam editar cada componente individualmente.

Cores estruturais

Token	HEX	Uso recomendado	Observações
`ocean-950`	`#020A16`	Fundo principal	Cor mais profunda; maior área visual.
`ocean-900`	`#061326`	Painéis e áreas de trabalho	Distingue a superfície principal do fundo.
`ocean-850`	`#091A31`	Cards e superfícies elevadas	Usar em grupos de conteúdo e componentes.
`ocean-800`	`#102641`	Hover, seleção suave e superfícies ativas	Não usar como fundo universal de todos os cards.

Cores de interação

Token	HEX	Uso recomendado
`blue-500`	`#2678FF`	Ação primária, seleção, foco e links de destaque
`blue-400`	`#48A5FF`	Destaques, ícones ativos e pequenos realces
`blue-selected`	`#163B70`	Fundo de seleção de intensidade moderada

Texto, bordas e estados semânticos

Token	HEX	Uso recomendado
`text-primary`	`#E8F1FF`	Títulos, conteúdo e texto principal
`text-secondary`	`#9CB4D4`	Descrições e metadados de importância média
`text-muted`	`#6985A9`	Dicas, horários e conteúdo auxiliar
`border-subtle`	`#173353`	Divisores e bordas discretas
`state-success`	`#35D39A`	Operação concluída ou estado positivo
`state-warning`	`#F4B84A`	Aviso, atenção ou limite relevante
`state-danger`	`#FF637D`	Falha, erro e ação destrutiva

Regras de aplicação da cor

1. O fundo deve ocupar a maior parte da tela. O azul de destaque não é uma cor de preenchimento universal.
2. Azul intenso significa ação, foco, seleção ou prioridade. Se todo elemento for azul, a cor deixa de orientar.
3. Verde, amarelo e vermelho têm semântica. Não devem ser usados como decoração aleatória.
4. A cor nunca deve ser o único indicador de estado. Combine-a com texto, ícone ou outro sinal perceptível.
5. Texto secundário precisa continuar legível. Em particular, metadados pequenos e controles desabilitados devem ser testados em contexto real.
6. Estados desabilitados podem ter menor ênfase, mas não devem desaparecer visualmente a ponto de confundir usuários.
7. Cores funcionais devem ser testadas em pares reais: texto sobre fundo, borda em superfície e foco contra o entorno.

1.3 Tipografia

A direção tipográfica combina uma família de interface geométrica e legível com uma família monoespaçada voltada a informações técnicas.

Manrope — interface

Uso: navegação, títulos, botões, labels, formulários, descrições e textos corridos. A Manrope deve oferecer uma aparência moderna e limpa sem tornar o conteúdo excessivamente informal.

JetBrains Mono — conteúdo técnico

Uso: código, comandos, logs, caminhos de arquivos, identificadores, diffs e outros conteúdos em que a largura consistente dos caracteres ajude a leitura. Não usar fonte monoespaçada para todos os parágrafos apenas para produzir uma aparência técnica: isso prejudica a leitura de texto comum.

Token	Tamanho de referência	Peso	Uso
`display`	28–32 px	700	Títulos de páginas especiais ou telas de entrada
`h1`	24 px	700	Título principal
`h2`	18 px	600–700	Seções
`h3`	15–16 px	600	Cards e subseções
`body`	14 px	400	Conteúdo padrão
`body-small`	13 px	400	Descrições e linhas secundárias
`label`	12 px	600	Rótulos de campo e pequenos títulos de controle
`caption`	11–12 px	400	Metadados, horários e contexto auxiliar
`code`	12–14 px	400	Código, comandos e trechos técnicos

Os valores são referências para desktop. Em janelas menores, ajuste a escala de acordo com a largura disponível, sem reduzir conteúdo essencial a tamanhos difíceis de ler.

Regras tipográficas

- Criar hierarquia primeiro com peso, espaçamento e cor; só depois aumentar o tamanho.
- Evitar parágrafos longos em fonte monoespaçada.
- Reservar caixa alta para rótulos pequenos e realmente úteis.
- Preservar quebras, indentação e caracteres especiais em código.
- Para texto corrido, mirar altura de linha entre 1,45 e 1,6; para código, em torno de 1,4, ajustando quando houver densidade elevada.
- Não depender somente de diferenças de cor para distinguir título, descrição e metadado.

1.4 Espaçamento e geometria

A escala utiliza uma unidade-base de **4 px**. Os valores derivados devem ser reutilizados em todo o sistema.

Token	Valor	Aplicação típica
`space-1`	4 px	Distância mínima entre ícone e texto compacto
`space-2`	8 px	Espaçamento interno compacto
`space-3`	12 px	Grupos de controles e itens relacionados
`space-4`	16 px	Padding padrão de componentes
`space-5`	20 px	Separação entre grupos

Token	Valor	Aplicação típica
`space-6`	24 px	Espaçamento entre seções
`space-8`	32 px	Separação ampla
`space-10`	40 px	Respiro estrutural
`space-12`	48 px	Separações de grande escala

Raios de borda

Token	Valor	Aplicação típica
`radius-sm`	6 px	Badges e chips compactos
`radius-md`	8 px	Botões e campos
`radius-lg`	12 px	Cards, grupos e painéis comuns
`radius-xl`	16 px	Modais e superfícies grandes

Evitar arredondamento excessivo em cada elemento. A geometria deve manter a sensação de ferramenta técnica, sem se tornar rígida ou agressiva.

1.5 Bordas, sombras e profundidade

A hierarquia deve ser construída principalmente com diferenças de superfície e bordas. Sombras luminosas são exceção, não padrão.

- **Nível 0 — Base:** fundo geral, sem sombra.
- **Nível 1 — Superfície:** painel ligeiramente distinto do fundo.
- **Nível 2 — Flutuante:** menus, popovers e caixas de sugestão com sombra escura.
- **Nível 3 — Foco:** diálogo ativo ou ação prioritária com borda azul e realce discreto.

Sombras de referência:

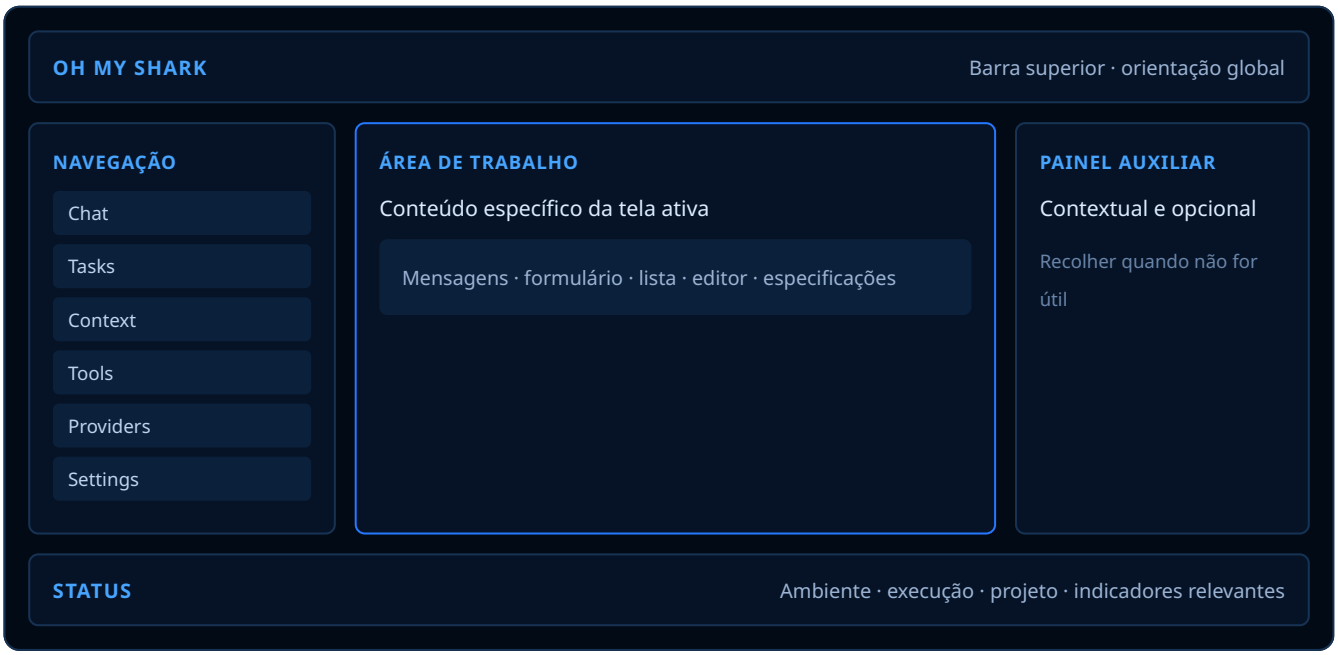
```
--shadow-panel: 0 8px 28px rgb(0 0 0 / 22%);
--shadow-dialog: 0 20px 64px rgb(0 0 0 / 42%);
--glow-focus: 0 0 0 2px rgb(38 120 255 / 20%);
```

O foco não deve depender do brilho. Ele precisa continuar reconhecível em telas sem efeitos, com contraste suficiente e, quando necessário, contorno claramente visível.

---

2. Arquitetura global da aplicação

2.1 Modelo de três áreas



A referência sugere três regiões: navegação à esquerda, área de trabalho central e painel contextual à direita. Essa é a composição de referência, não uma obrigação para todos os módulos.

Barra superior

Responsabilidade: orientação global e acesso rápido a controles do aplicativo.

Elementos possíveis:

- Identidade visual do Oh My Shark.
- Projeto, diretório ou espaço de trabalho atual.
- Ações globais, como alternar painéis ou abrir preferências.
- Indicadores compactos de atividade, quando relevantes.

Dimensões de referência: 36–44 px de altura; ícones de 16–18 px; divisor inferior discreto.

Evitar repetir informações que já aparecem de forma clara na barra de status. Controles contextuais só devem aparecer quando houver ação aplicável.

Navegação lateral

Responsabilidade: organizar os módulos principais e conservar uma orientação estável entre as telas.

Larguras de referência:

- Expandida: 280–292 px.
- Recolhida: 64–72 px.

Itens de navegação:

- Altura de 36–40 px.
- Ícones de 16–18 px.
- Espaçamento horizontal de 10–12 px.
- Seleção com fundo azul discreto e texto claro.
- Atalhos alinhados à direita, quando existirem.

A seleção deve possuir mais de uma indicação visual, sem depender apenas da cor. Ao recolher a barra, os rótulos podem ser escondidos, mas os ícones precisam manter nomes acessíveis e tooltips úteis.

Área de trabalho central

É a região prioritária. Deve adaptar sua composição ao tipo de tarefa:

- O chat funciona como fluxo contínuo.
- Configurações funcionam melhor como formulários organizados em seções.
- Especificações podem utilizar tabelas, navegação por tópicos e blocos de código.
- Catálogos podem usar listas ou grades conforme a densidade dos dados.

Regras comuns: cabeçalho com título e ações da página, rolagem independente, conteúdo com largura de leitura confortável, estados de interface completos e espaçamento uniforme.

Painel contextual direito

Responsabilidade: apresentar informações auxiliares associadas ao trabalho atual, por exemplo, arquivos relacionados, eventos de execução, contexto ou detalhes de um item selecionado.

Largura de referência: 280–320 px. O painel deve ser opcional, recolhível e contextual. Cards independentes podem aparecer ou desaparecer sem deixar lacunas. Se não houver conteúdo útil, o painel pode permanecer oculto.

Não presumir que sempre existam tarefas, arquivos, modelos, ferramentas ou estatísticas ativos.

Barra de status inferior

Pode apresentar projeto ou diretório ativo, estado de execução, branch Git quando houver integração, conexão ou ambiente ativo e indicadores operacionais relevantes.

A barra não deve se tornar um depósito de metadados. Só deve incluir informações úteis naquele contexto, atualizadas a partir de fontes reais.

2.2 Responsividade

Breakpoints iniciais sugeridos:

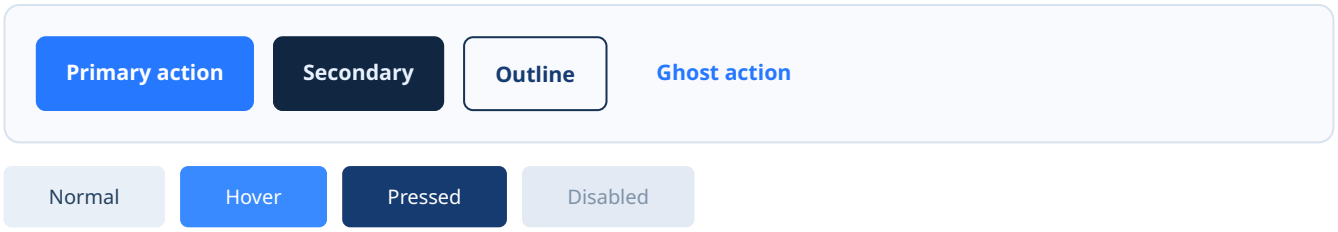
Largura disponível	Comportamento esperado
1280 px ou mais	Layout completo, com painéis conforme a necessidade
1024–1279 px	Painel contextual recolhível; área central priorizada
768–1023 px	Navegação compacta e painéis auxiliares sob demanda
Abaixo de 768 px	Uma área principal, navegação em drawer e painéis próprios ou overlays

Esses intervalos são pontos de partida. A decisão final deve ser baseada na largura mínima necessária para os controles e no conteúdo real de cada tela, e não apenas no tamanho físico do dispositivo.

---

3. Componentes fundamentais

3.1 Botões



Variantes recomendadas:

- **Primary:** ação principal da seção, com fundo azul.
- **Secondary:** ação complementar com superfície azul-marinho.
- **Outline:** ação de menor prioridade delimitada por borda.
- **Ghost:** ação leve, geralmente textual ou iconográfica.
- **Destructive:** ação que remove dados ou produz efeitos relevantes; deve usar a semântica de perigo e confirmação proporcional ao risco.

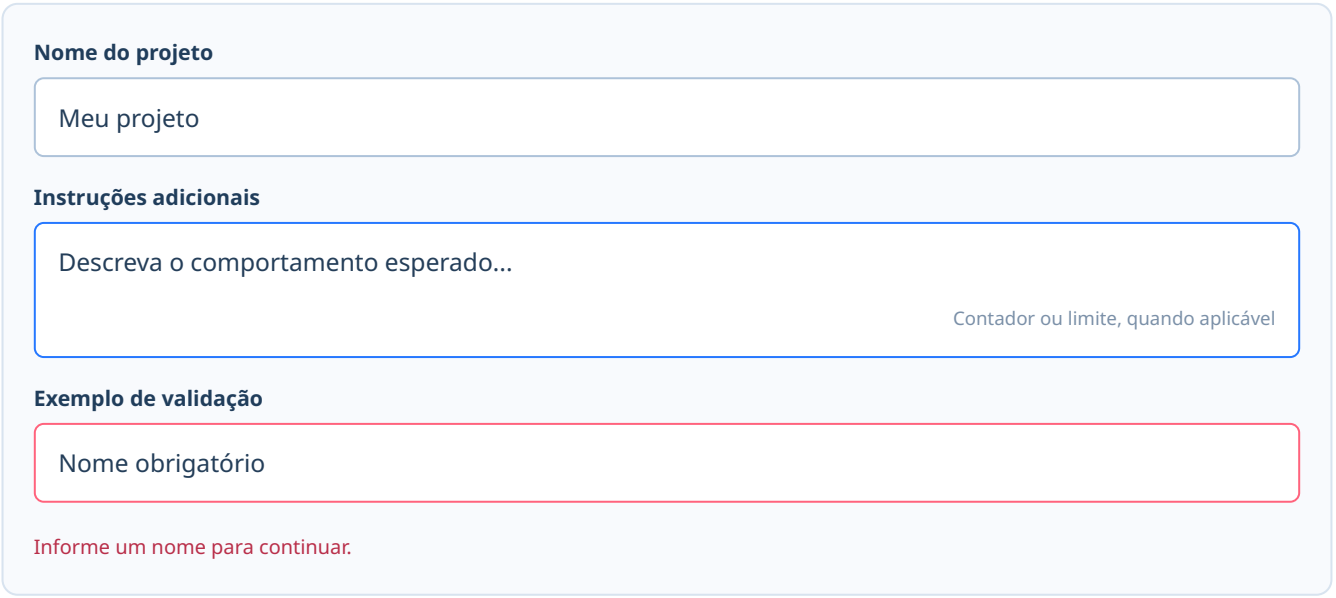
Dimensões de referência: 28–32 px para compacto, 36 px para padrão e 40–44 px para ações maiores. Padding horizontal de 12–16 px, raio de 8 px e ícone de 16 px com distância de aproximadamente 8 px do rótulo.

Estados

Cada botão deve definir estados normal, hover, pressed, foco de teclado, loading e disabled quando aplicáveis. O estado loading precisa impedir envios duplicados quando necessário e informar que a operação está em andamento. Disabled deve comunicar indisponibilidade sem desaparecer.

As ações primárias devem ser escassas. Uma tela com cinco botões primários concorrentes não possui uma hierarquia útil.

3.2 Campos de entrada



Altura de campo simples: 36–40 px. Textareas devem começar, em geral, entre 88–120 px, com crescimento controlado. Cada campo precisa de label; placeholder é uma dica, não substitui o rótulo.

Regras:

- Foco por borda azul e realce discreto.
- Erro acompanhado de mensagem próxima, explícita e acionável.
- Código em fonte monoespaçada.
- Campos alterados e ainda não salvos identificáveis.
- Mensagens de validação associadas ao campo correspondente.
- Valores longos não devem alargar a tela; devem ter quebra, truncamento com acesso ao conteúdo integral ou rolagem horizontal quando isso fizer sentido.

3.3 Cards e painéis

Cards são indicados para informações independentes: ferramentas, provedores, grupos de configuração, tarefas e resumos. Não devem ser usados indiscriminadamente em conteúdo que funciona melhor como fluxo contínuo, como o histórico do chat.

Regras de referência:

- Padding interno: 16 px.
- Espaço interno entre elementos: 8–12 px.
- Título: 14–16 px.
- Descrição curta, com expansão se o conteúdo exigir.
- Ações secundárias agrupadas em menu contextual quando isso reduzir a poluição.
- Sem sombra intensa ou borda azul permanente em todos os cards.

3.4 Listas, tabelas e árvores

Use listas para itens de natureza semelhante; use tabelas quando a comparação entre valores ou atributos for central.

- Altura de linha de lista: 36–44 px, de acordo com a densidade.
- Cabeçalhos de tabela claramente distinguíveis.
- Alinhamento consistente de valores, estados e ações.
- Linhas selecionadas com fundo discreto.
- Árvores de arquivos com indentação uniforme e expansão acessível.
- Truncamento controlado para conteúdo extenso, com acesso ao texto completo.
- Ordenação, filtro e busca só devem ser mostrados quando agregarem valor.

3.5 Ícones

Adotar uma única família, preferencialmente Lucide, para preservar a consistência de traço e proporção.

Tamanho	Uso
14 px	Metadados e controles compactos
16 px	Botões e listas
18 px	Navegação principal
20 px	Ações de destaque
24 px	Estados vazios e ilustrações simples

Ícones incomuns devem possuir tooltip e, sempre que possível, rótulo textual. Botões apenas com ícones precisam de nome acessível. Evitar misturar bibliotecas com metáforas, espessuras ou alinhamentos diferentes.

3.6 Estados de interface

Toda tela que carrega dados ou executa operações deve definir pelo menos os estados pertinentes entre:

- **Carregando:** skeleton ou indicador de atividade sem mudanças bruscas no layout.
- **Vazio:** explica o que falta e sugere uma ação pertinente.
- **Sucesso:** confirma uma operação sem bloquear o fluxo.
- **Atenção:** explica uma condição que exige cuidado.
- **Erro:** descreve o problema e o próximo passo conhecido.
- **Desconectado:** diferencia ausência de conexão de falha geral.
- **Permissão insuficiente:** informa que a ação não está autorizada, sem sugerir que ela ocorreu.
- **Em andamento:** indica a atividade atual e oferece interrupção quando suportada.
- **Cancelado:** confirma que o cancelamento aconteceu.
- **Parcialmente disponível:** identifica o que está acessível e o que não está.

A linguagem deve ser concreta. Evitar mensagens genéricas como “Algo deu errado” quando a aplicação pode explicar o que aconteceu e o que o usuário pode fazer.

4. Especificação das telas

4.1 Chat

Objetivo

Oferecer um espaço contínuo para conversar com o agente, delegar tarefas, acompanhar ações e ler resultados. A tela pode conter texto, código, anexos, diffs, logs e resultados de ferramentas, conforme os recursos efetivamente implementados.

Estrutura

1. Cabeçalho com nome ou contexto da conversa e ações pertinentes.
2. Histórico de mensagens e blocos ricos.
3. Indicadores temporários de atividade, quando úteis.
4. Compositor de mensagem fixado à base da área de trabalho.
5. Acessórios de entrada, como anexos, contexto, ferramentas, modelos e opções, somente quando disponíveis.

Regras específicas

- Mensagens do usuário e do agente devem ser distinguíveis, mas pertencer à mesma linguagem visual.
- Código, logs, diffs e resultados de ferramentas precisam de componentes próprios.
- Mensagens extensas devem manter largura confortável para leitura.
- O compositor cresce até um limite definido e então permite rolagem interna.
- Controles refletem a disponibilidade real dos recursos.
- O histórico não deve virar uma sequência de cards independentes.
- Uma conversa vazia deve ter um estado inicial útil, com sugestões opcionais e sem conteúdo fictício.
- Em execuções longas, separar a atividade atual, as ferramentas executadas e a resposta final.
- Ações de interromper, repetir e continuar só aparecem quando forem suportadas.

Casos a testar

Mensagem de uma linha, mensagem muito longa, código extenso, anexo, falha de ferramenta, cancelamento, resposta em andamento, chat vazio, painel lateral oculto e janela estreita.

4.2 Tasks — tarefas e execuções

Objetivo

Acompanhar trabalhos delegados ao agente, operações em andamento e atividades que precisam de supervisão.

Estrutura

- Cabeçalho com título, filtros e ações disponíveis.
- Resumo opcional de estados, sem estatísticas artificiais.
- Lista de tarefas ou execuções.
- Detalhes do item selecionado: etapas, eventos, resultados e erros.

- Controles de retomada, cancelamento ou repetição, apenas se a plataforma os suportar.

Estados recomendados

Aguardando, em execução, concluída, falhou, cancelada, interrompida e aguardando intervenção. Nem todo produto precisará de todos os estados; os rótulos devem corresponder ao modelo real de execução.

Não misturar o estado da tarefa com o estado de uma ferramenta ou provedor. Uma execução pode falhar mesmo que a integração esteja conectada. Indicadores percentuais só devem aparecer quando o sistema puder medir o progresso; em tarefas sem métrica confiável, use etapas qualitativas.

4.3 Context — contexto de trabalho

Objetivo

Visualizar, selecionar e administrar informações que podem influenciar o trabalho do agente.

Estrutura

- Seletor de escopo, quando houver múltiplos espaços de trabalho.
- Lista de fontes, arquivos ou instruções.
- Busca e filtros, se necessários.
- Painel de visualização do item selecionado.
- Informações de origem, atualização e disponibilidade.
- Ações de inclusão, remoção ou ativação, conforme as capacidades reais.

Regra de integridade

É preciso distinguir três condições: **contexto disponível**, **contexto selecionado** e **contexto efetivamente utilizado**. Elas podem não ser equivalentes. A interface não deve induzir o usuário a acreditar que uma fonte foi aplicada quando foi apenas selecionada ou estava disponível no catálogo.

4.4 Tools — ferramentas

Objetivo

Oferecer um catálogo de ferramentas e seus controles, incluindo requisitos, parâmetros e permissões.

Estrutura

- Lista ou grade de ferramentas.
- Busca e categorias, caso o catálogo seja extenso.
- Detalhes da ferramenta selecionada.
- Parâmetros, permissões e configurações.
- Estado de disponibilidade.
- Histórico de utilização ou resultados, se suportado.

Cada ferramenta precisa de uma descrição clara, identificação de requisitos e apresentação de riscos relevantes. Operações com efeitos externos ou que alteram arquivos devem deixar evidente o que será executado e respeitar os mecanismos reais de autorização.

Não mostrar controles como disponíveis quando a integração não existe. Uma ferramenta pode estar instalada, habilitada, autorizada ou em uso: esses estados devem ser representados de forma precisa, sem tratá-los como sinônimos.

4.5 Providers — provedores e modelos

Objetivo

Configurar conexões com serviços de IA e administrar modelos disponibilizados por essas conexões.

Estrutura

- Catálogo de provedores.
- Detalhes e configuração de cada provedor.
- Credenciais e variáveis necessárias.
- Teste de conexão e resultado.
- Catálogo de modelos, se disponível.
- Parâmetros específicos de cada modelo.
- Seleção de modelo padrão, caso esse conceito exista no produto.

Estados que não devem ser confundidos

- **Configurado:** os dados necessários foram registrados.
- **Conectado:** uma verificação recente confirmou conexão.
- **Desconectado:** não há conexão ativa ou verificável.
- **Testando:** a aplicação está realizando uma verificação.
- **Erro de autenticação:** a conexão foi recusada por credenciais ou autorização.
- **Quota atingida:** o serviço informou limite de uso.
- **Indisponibilidade temporária:** a falha não é necessariamente de configuração.

Não revelar segredos completos. Se a conexão ainda não foi testada, evitar apresentar “conectado” como fato. Mensagens de erro devem exibir informação suficiente para orientar o usuário, sem expor tokens ou dados sensíveis.

4.6 Settings — configurações gerais

Objetivo

Concentrar preferências globais e opções de comportamento sem duplicar integralmente telas especializadas.

Categorias sugeridas

- Interface: tema, densidade, navegação e animações.
- Comportamento: preferências gerais do agente.
- Execução: opções de funcionamento, quando suportadas.
- Privacidade e segurança: controles realmente disponíveis ao usuário.
- Atalhos: combinações de teclas e ações correspondentes.
- Integrações: acessos às telas de provedores e ferramentas.

Comportamento

- Agrupar opções por finalidade.
- Descrever consequências de opções avançadas.
- Diferenciar configurações salvas automaticamente das que exigem confirmação.
- Mostrar estado de salvamento e falhas de persistência.
- Oferecer restauração de valores padrão quando aplicável.
- Exigir confirmação proporcional ao risco para redefinições destrutivas.

Uma configuração alterada localmente, mas ainda não salva, deve ser distinguida de uma configuração já persistida. Não apresentar sucesso até que o armazenamento confirme a operação.

4.7 System Prompt — prompt de sistema

Objetivo

Editar, validar e administrar as instruções persistentes que orientam o comportamento do agente. A tela é um editor técnico, não um chat comum.

Estrutura proposta

1. Cabeçalho com escopo da configuração e estado de salvamento.
2. Editor principal em texto simples ou Markdown, conforme o suporte do produto.
3. Navegação por seções ou outline para documentos extensos.
4. Ferramentas de busca, contagem de caracteres e visualização Markdown, quando suportadas.
5. Painel de validação para problemas de formato ou inconsistências.
6. Ações de salvar, descartar e, se houver suporte, restaurar versão anterior.

Requisitos de comportamento

- Distinguir prompt salvo de rascunho.
- Preservar quebras de linha, indentação e caracteres especiais.
- Informar se o escopo é global, por projeto, por agente ou por sessão.
- Não afirmar que um prompt está ativo sem confirmação de que foi aplicado no fluxo de execução.
- Se instruções de diferentes origens forem combinadas, mostrar a composição efetiva quando tecnicamente possível.
- Não executar o texto como código por padrão.
- Não expor segredos ou conteúdo sensível de forma desnecessária.

O editor deve ser o elemento dominante. Metadados e validação são auxiliares e podem ser recolhidos. Para prompts grandes, considerar busca, navegação por cabeçalhos, indicação de alterações não salvas e prevenção de perda de conteúdo.

4.8 Specifications — especificações técnicas

Objetivo

Funcionar como um centro de documentação e inspeção técnica do projeto, distinto das configurações operacionais.

Categorias possíveis

- Visão geral e escopo.
- Arquitetura e módulos.
- Requisitos funcionais.
- Requisitos não funcionais.
- Interface e design system.
- Integrações e APIs.
- Segurança e permissões.
- Restrições, dependências e decisões técnicas.
- Histórico de alterações.

Recursos visuais

- Navegação por seções.
- Busca no documento.
- Blocos de código com destaque de sintaxe.
- Tabelas de parâmetros e contratos.
- Diagramas quando houver representação estruturada disponível.
- Links internos entre requisitos e módulos.
- Metadados de versão e origem.
- Modo de leitura e edição, se suportados.

Regra de integridade

As especificações precisam diferenciar itens oficiais, propostos, desatualizados e não verificados conforme os dados reais. Requisitos implementados não podem ser misturados com requisitos planejados sem indicar a diferença. A interface não deve inventar versões, datas ou estados de validação.

---

5. Navegação e consistência entre telas

A aplicação deve parecer um produto coeso mesmo quando os módulos utilizam padrões de conteúdo diferentes.

Elemento	Regra compartilhada
Cabeçalho	Título, contexto e ações da página
Navegação	Seleção persistente e atalhos coerentes
Botões	Alturas, raios e estados consistentes
Formulários	Labels, foco, validação e erros uniformes
Cards	Espaçamentos, bordas e hierarquia compartilhados
Feedback	Semântica e posicionamento previsíveis

Elemento	Regra compartilhada
Painel direito	Opcional, contextual e recolhível
Barra de status	Só informações globais realmente úteis
Diálogos	Foco, ações e fechamento consistentes
Busca	Comportamento uniforme de entrada, resultado e vazio

5.1 Elementos permanentes, condicionais e dinâmicos

- **Permanentes:** identidade do produto, navegação principal e acesso a configurações globais.
- **Condicionais:** recursos disponíveis somente em certas instalações, projetos ou contextos.
- **Dinâmicos:** nomes, mensagens, modelos, resultados, metadados e estados de execução.
- **Contextuais:** ações válidas somente em determinadas situações.

Um chat sem ferramentas, uma sessão sem tarefas ou um projeto sem arquivos deve continuar apresentando uma interface completa. Quando um módulo não estiver disponível, a aplicação deve explicar a condição ou ocultar a entrada de acordo com a regra do produto. Não deixar espaços vazios sem função.

5.2 Atalhos de teclado

Combinações propostas, a validar durante a implementação:

- `Ctrl/Cmd + K`: busca ou comando global.
- `Ctrl/Cmd + N`: novo item no módulo atual, quando aplicável.
- `Ctrl/Cmd + ,`: configurações.
- `Esc`: fechar menu ou diálogo e cancelar interação reversível quando apropriado.
- `Ctrl/Cmd + Enter`: enviar ou confirmar em campos específicos, quando fizer sentido.

Esses atalhos são exemplos de convenção, não afirmações de que já existam. Devem evitar conflitos com o sistema operacional, ser documentados e, sempre que possível, permitir personalização. `Esc` não deve descartar alterações irreversivelmente sem confirmação ou salvaguarda.

---

6. Microinterações e animações

As animações devem comunicar mudanças de estado, hierarquia e continuidade. Não devem transformar cada clique em uma demonstração visual nem atrasar o trabalho.

Interação	Duração sugerida	Comportamento
Hover e foco	120–160 ms	Alteração suave de fundo, borda ou opacidade
Painéis e navegação	180–240 ms	Deslocamento curto e previsível
Feedback de operação	Breve e não bloqueante	Confirmação visual curta

Interação	Duração sugerida	Comportamento
Processamento	Enquanto necessário	Indicador discreto e etapa atual quando conhecida

Curva de transição sugerida: `cubic-bezier(0.2, 0, 0, 1)`.

Regras de movimento

- Preferir animações de opacidade e transformações em vez de alterações que recalculam o layout continuamente.
- Não animar componentes estáticos sem propósito.
- Não usar pulsação constante como único sinal de atividade.
- Não usar brilho como indicador exclusivo de foco ou execução.
- Respeitar `prefers-reduced-motion`.
- Manter controles utilizáveis durante transições e não bloquear ações por efeitos decorativos.
- Quando uma ação demorar, oferecer feedback de atividade fundamentado no estado real.

---

7. Acessibilidade e ergonomia

Adotar **WCAG 2.2 nível AA** como referência de projeto e validar a implementação em diferentes larguras e modos de interação.

Requisitos mínimos:

1. Todos os controles devem possuir foco de teclado visível.
2. A ordem de foco precisa ser lógica e acompanhar a estrutura da tela.
3. Controles iconográficos precisam de nomes acessíveis.
4. Estados de erro, sucesso e atividade não podem depender exclusivamente da cor.
5. Campos devem possuir labels e mensagens de validação associadas.
6. Diálogos devem controlar o foco corretamente e devolvê-lo ao elemento de origem quando fechados.
7. A interface deve continuar utilizável com zoom e redimensionamento do texto.
8. Interações por hover devem ter alternativa equivalente.
9. Contraste deve ser testado em texto, ícones, bordas funcionais e estados de foco.
10. Animações devem respeitar a preferência de redução de movimento do sistema operacional.

Ergonomia de uso contínuo

O produto pode ser utilizado durante períodos prolongados. Isso torna importante reduzir estímulos desnecessários, manter hierarquia previsível, evitar mensagens intrusivas e preservar o estado do trabalho quando painéis são recolhidos ou a janela é redimensionada.

Não sacrificar legibilidade para mostrar mais linhas na tela. Oferecer densidade compacta como opção pode ser útil, mas o modo padrão deve ser confortável. Alvos interativos devem possuir área suficiente e estar claramente separados de ações destrutivas.

---

8. Especificações técnicas de implementação

8.1 Arquitetura em camadas de tokens

Os tokens devem separar valores de referência, significados semânticos e componentes:

1. **Primitivos:** cores brutas, espaçamento, tipografia, raios e durações.
2. **Semânticos:** fundo principal, texto secundário, borda de foco, sucesso, perigo e superfície elevada.
3. **Componentes:** botões, campos, navegação, cards, diálogos, mensagens e painéis.

Essa separação permite alterar o tema sem editar cada componente individualmente. Componentes devem consumir principalmente tokens semânticos, e não codificar cores brutas diretamente.

8.2 Tokens CSS de referência

```
:root {
  /* Color primitives */
  --ocean-950: #020A16;
  --ocean-900: #061326;
  --ocean-850: #091A31;
  --ocean-800: #102641;
  --blue-500: #2678FF;
  --blue-400: #48A5FF;

  /* Semantic colors */
  --color-bg: var(--ocean-950);
  --color-surface: var(--ocean-900);
  --color-surface-raised: var(--ocean-850);
  --color-surface-hover: var(--ocean-800);
  --color-text: #E8F1FF;
  --color-text-secondary: #9CB4D4;
  --color-text-muted: #6985A9;
  --color-border: #173353;
  --color-focus: var(--blue-500);
  --color-primary: var(--blue-500);
  --color-success: #35D39A;
  --color-warning: #F4B84A;
  --color-danger: #FF637D;

  /* Typography */
  --font-ui: "Manrope", sans-serif;
  --font-code: "JetBrains Mono", monospace;
  --text-xs: 12px;
  --text-sm: 13px;
  --text-md: 14px;
  --text-lg: 16px;
```

```
  --text-xl: 18px;
  --text-2xl: 24px;

  /* Spacing */
  --space-1: 4px;
  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-6: 24px;
  --space-8: 32px;

  /* Geometry */
  --radius-sm: 6px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-xl: 16px;

  /* Motion */
  --duration-fast: 140ms;
  --duration-normal: 200ms;
  --duration-slow: 240ms;
}
```

Esse trecho é uma base inicial, não uma biblioteca completa. Estados de hover, pressed, loading e disabled, além de temas alternativos ou de alto contraste, devem ser definidos conforme as necessidades reais.

8.3 Organização de arquivos sugerida

```
src/
├── design-system/
│   ├── tokens/
│   │   ├── colors.css
│   │   ├── typography.css
│   │   ├── spacing.css
│   │   └── motion.css
│   ├── components/
│   │   ├── Button/
│   │   ├── Input/
│   │   ├── Select/
│   │   ├── Badge/
│   │   ├── Card/
│   │   ├── Dialog/
│   │   ├── Tooltip/
│   │   ├── DataTable/
│   │   └── EmptyState/
│   └── patterns/
│       ├── AppShell/
│       ├── PageHeader/
│       ├── InspectorPanel/
```

```
│       └── StatusBar/
├── features/
│   ├── chat/
│   ├── tasks/
│   ├── context/
│   ├── tools/
│   ├── providers/
│   ├── settings/
│   ├── system-prompt/
│   └── specifications/
└── app/
    └── routes/
```

A organização é sugestiva e não impõe framework. Os nomes, pastas e responsabilidades devem ser adaptados ao repositório. O importante é separar tokens e componentes reutilizáveis dos módulos específicos.

8.4 Princípios de arquitetura

- **Componentes compartilhados:** elementos repetidos devem ter uma implementação canônica.
- **Conteúdo desacoplado:** mensagens, modelos, ferramentas e sessão são dados, não partes fixas do layout.
- **Composição modular:** cada página combina componentes conforme sua finalidade.
- **Estados explícitos:** loading, empty, success, error, disabled e active são tratados de maneira consistente.
- **Persistência controlada:** preferências persistentes não se confundem com estado temporário da interface.
- **Permissões respeitadas:** controles refletem capacidades realmente disponíveis.
- **Layouts adaptáveis:** painéis podem ser recolhidos sem quebrar o conteúdo.
- **Design testável:** tokens, componentes e estados possuem critérios verificáveis.

---

9. Critérios de qualidade e validação

O sistema só pode ser considerado completo quando houver maneiras objetivas de verificar a implementação.

Critério	O que verificar
Consistência	Componentes equivalentes possuem aparência e comportamento equivalentes
Responsividade	Nenhuma tela depende obrigatoriamente de três painéis simultâneos
Conteúdo dinâmico	Textos longos, listas vazias e dados ausentes não quebram o layout
Acessibilidade	Teclado, foco, contraste e nomes acessíveis funcionam
Estados	Carregamento, falhas, desconexões e ausência de dados são contemplados
Segurança visual	Credenciais e dados sensíveis não são expostos indevidamente
Interação	Botões, atalhos e menus executam ações reais

Critério	O que verificar
Performance	Animações não prejudicam a fluidez
Manutenção	Mudanças nos tokens propagam-se corretamente
Fidelidade	A identidade permanece reconhecível em todas as telas

9.1 Checklist de aceite

- ☐ Todas as telas utilizam a mesma escala de cores, tipografia e espaçamento.
- ☐ Chat, Tasks, Context, Tools, Providers e Settings estão cobertos.
- ☐ System Prompt e Specifications possuem estruturas específicas.
- ☐ O painel contextual pode ser ocultado sem quebrar a composição.
- ☐ A navegação não depende de modelos, ferramentas ou tarefas específicos.
- ☐ Componentes possuem os estados aplicáveis: normal, hover, foco, pressed, loading e disabled.
- ☐ Estados vazios, de desconexão e de erro foram projetados.
- ☐ A interface funciona em diferentes larguras de janela.
- ☐ Texto e contraste foram testados.
- ☐ Não há informações fictícias apresentadas como dados reais.
- ☐ Nenhuma funcionalidade existente foi removida apenas para adequar a aparência.
- ☐ A configuração salva é distinguível de um rascunho ainda não persistido.
- ☐ A interface reflete os estados reais das integrações e operações.

10. Direção final

O princípio que deve orientar todas as decisões é simples: **o Oh My Shark deve parecer uma ferramenta de desenvolvimento profissional, não um dashboard decorativo.**

Isso implica priorizar a área de trabalho, tornar a navegação previsível, usar o azul para orientar a atenção e reservar o espaço auxiliar para informações úteis à tarefa atual. A identidade deve continuar reconhecível quando o usuário troca de modelo, muda de projeto, abre uma ferramenta diferente, não possui tarefas ativas ou usa apenas uma parte dos recursos.

O resultado esperado é um sistema visual único, mas flexível: consistente nos fundamentos, especializado em cada tela e neutro em relação ao conteúdo que pode mudar com o uso.

Decisões que ainda precisam de validação no produto

Antes de transformar esta proposta em implementação definitiva, validar contra o repositório real:

1. Quais módulos existem de fato e quais são opcionais.
2. Quais ações, estados e integrações são realmente suportados.
3. Como o aplicativo armazena preferências, prompts, contexto e credenciais.
4. Quais requisitos de segurança e autorização governam ferramentas e operações externas.
5. Qual framework e sistema de componentes serão utilizados.
6. Se serão oferecidos outros temas além de Dark Ocean.
7. Quais breakpoints e densidades funcionam melhor nos cenários de uso reais.

Este design system especifica a linguagem e o comportamento esperado da interface; ele não afirma que todas as capacidades descritas já estejam implementadas.

Fim do documento — Oh My Shark Design System, edição 1.0.